Experiment 9

AIM: Use pytest to test python application

1. Objective

- Understand unit testing concepts
- Write test cases using PyTest
- Execute and analyze test results
- Handle edge cases and exceptions
- Use parameterized testing

2. Prerequisites

- Python
- PyTest

3. Installation

Install pytest using pip:

pip install pytest

Verify it :

pytest –version

4. Create a Python module with basic mathematical operations and write PyTest test cases to validate:

- Correct outputs
- Edge cases
- Exception handling

Step 1: Create Project PythonDemo and add file pythondemo.py

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
def is_even(num):
    return num % 2 == 0
```

Step 2: Create a test file test_pythondemo.py

```
import pytest
import pythondemo
# Passing test
def test_add():
    assert pythondemo.add(2, 3) == 5
# Failing test
def test_multiply_fail():
    assert pythondemo.multiply(4, 3) == 11
# Exception test
def test_divide_by_zero():
    with pytest.raises(ValueError):
        pythondemo.divide(10, 0)
```

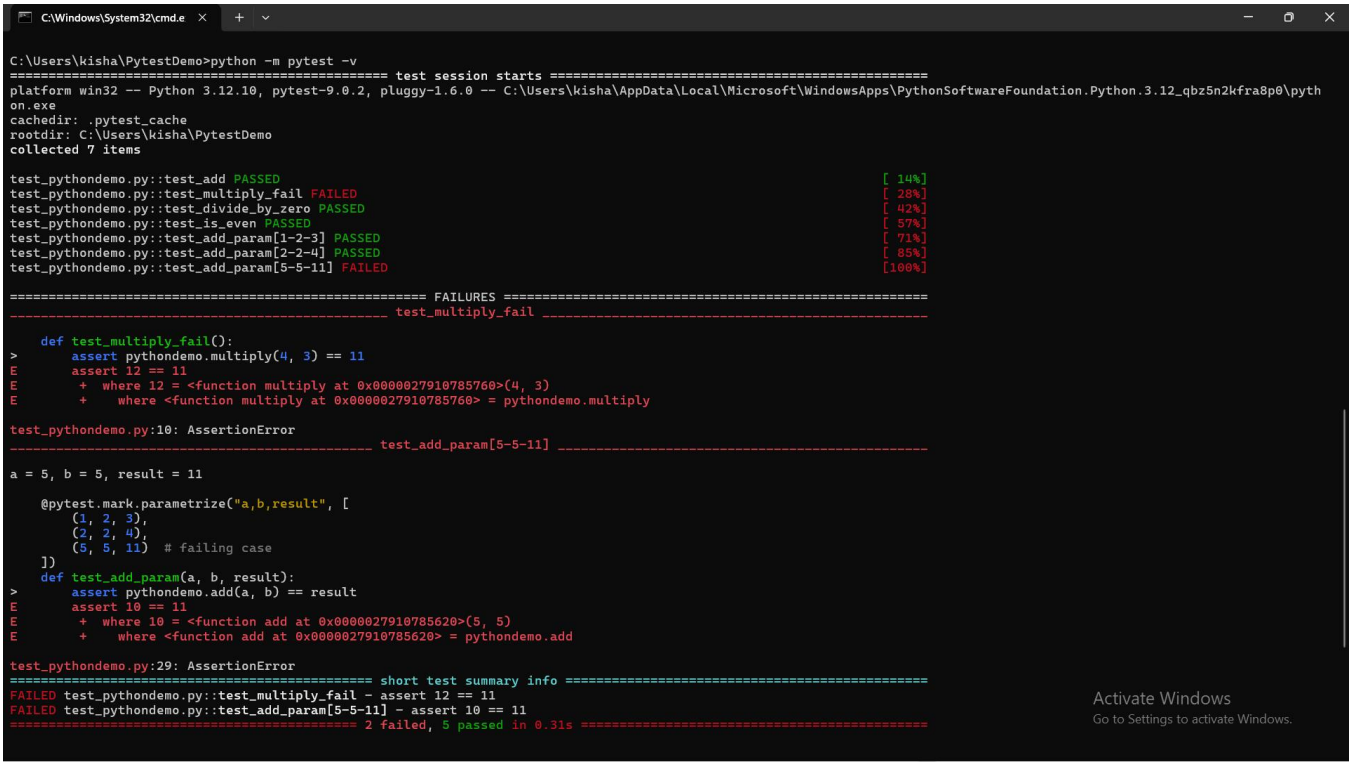
```
# Logical test
def test_is_even():
    assert pythondemo.is_even(4) is True
    assert pythondemo.is_even(5) is False


# Parametrized test
@pytest.mark.parametrize("a,b,result", [
    (1,2,3),
    (2,2,4),
    (5,5,11) # failing case
])
def test_add_param(a,b,result):
    assert pythondemo.add(a,b) == result
```

5. Execution

Open terminal in that folder

pytest –v



Task for you:

Task 1:

Add a function:

```
def factorial(n):
```

Write test cases.

pythondemo.py

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
```

```
        raise ValueError("Cannot divide by zero")
    return a / b
def is_even(num):
    return num % 2 == 0
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Task 2:

Write test cases for:

Negative numbers

Large inputs

test_pythondemo.py

```
import pytest
import pythondemo
def test_add():
    assert pythondemo.add(2, 3) == 5


def test_divide_by_zero():
    with pytest.raises(ValueError):
        pythondemo.divide(10, 0)
```

```
def test_is_even():
    assert pythondemo.is_even(4) is True
    assert pythondemo.is_even(5) is False
@pytest.mark.parametrize("n, expected", [
    (0, 1),
    (1, 1),
    (5, 120),
])
def test_factorial_standard(n, expected):
    assert pythondemo.factorial(n) == expected

def test_factorial_negative():
    with pytest.raises(ValueError):
        pythondemo.factorial(-5)

def test_factorial_large_input():
    assert pythondemo.factorial(20) == 2432902008176640000

def test_multiply_fail():
    assert pythondemo.multiply(4, 3) == 11

def test_add_fail():
    assert pythondemo.add(2, 2) == 5

def test_factorial_fail():
    assert pythondemo.factorial(4) == 20
```

```
C:\Users\kisha\PytestDemo>python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\kisha\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\pyth
on.exe
cachedir: .pytest_cache
rootdir: C:\Users\kisha\PytestDemo
collected 11 items

test_pythondemo.py::test_add PASSED                                      [  9%]
test_pythondemo.py::test_divide_by_zero PASSED                           [ 18%]
test_pythondemo.py::test_is_even PASSED                                  [ 27%]
test_pythondemo.py::test_factorial_standard[0-1] PASSED                  [ 36%]
test_pythondemo.py::test_factorial_standard[1-1] PASSED                  [ 45%]
test_pythondemo.py::test_factorial_standard[5-120] PASSED                [ 54%]
test_pythondemo.py::test_factorial_negative PASSED                       [ 63%]
test_pythondemo.py::test_factorial_large_input PASSED                    [ 72%]
test_pythondemo.py::test_multiply_fail FAILED                            [ 81%]
test_pythondemo.py::test_add_fail FAILED                                 [ 90%]
test_pythondemo.py::test_factorial_fail FAILED                           [100%]
```

```
================================== FAILURES ===================================
_____________________________ test_multiply_fail ______________________________

    def test_multiply_fail():
>       assert pythondemo.multiply(4, 3) == 11   # actual is 12
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 12 == 11
E        +  where 12 = <function multiply at 0x000001EA12E096C0>(4, 3)
E        +    where <function multiply at 0x000001EA12E096C0> = pythondemo.multiply

test_pythondemo.py:40: AssertionError
_____________________________ test_add_fail ______________________________

    def test_add_fail():
>       assert pythondemo.add(2, 2) == 5   # actual is 4
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 4 == 5
E        +  where 4 = <function add at 0x000001EA12E09580>(2, 2)
E        +    where <function add at 0x000001EA12E09580> = pythondemo.add

test_pythondemo.py:44: AssertionError
_____________________________ test_factorial_fail ______________________________

    def test_factorial_fail():
>       assert pythondemo.factorial(4) == 20   # actual is 24
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       assert 24 == 20
E        +  where 24 = <function factorial at 0x000001EA12E098A0>(4)
E        +    where <function factorial at 0x000001EA12E098A0> = pythondemo.factorial

test_pythondemo.py:48: AssertionError
=========================== short test summary info ===========================
FAILED test_pythondemo.py::test_multiply_fail - assert 12 == 11
FAILED test_pythondemo.py::test_add_fail - assert 4 == 5
FAILED test_pythondemo.py::test_factorial_fail - assert 24 == 20
========================= 3 failed, 8 passed in 0.32s =========================
```